

PERN + Prisma ORM

Complete Learning Guide

PostgreSQL • Express • React • Node.js • Prisma ORM

Tayab — DevAndDone | June 2026

Contents

1. PostgreSQL — Database kya hai aur kaise kaam karta hai
2. Prisma ORM — Kya hai, kyun use karte hain
3. Project Setup — Zero se shuru
4. Schema aur Models — Blueprint banana
5. Migrations — DB changes safely apply karna
6. Prisma CRUD Queries — Complete Reference
7. Express + Prisma — REST API banana
8. MVC Architecture — Production folder structure
9. Error Handling — Production-grade
10. Input Validation — Security layer
11. Relations — One-to-Many, Many-to-Many
12. Advanced Prisma — Filtering, Sorting, Pagination
13. Authentication — JWT + Prisma
14. Environment Variables — Best Practices
15. Interview Preparation — Q&A;

1. PostgreSQL — Database kya hai

PostgreSQL ek relational database hai — matlab data tables mein store hota hai, jaise Excel sheets. Har table mein rows (records) aur columns (fields) hote hain. Tables aapas mein relations se connected hoti hain.

■ Core Concepts

Concept	Matlab	Example
Database	Ek project ka container	taskmanager
Table	Ek entity ka data	User, Task, Order
Row	Ek record	{ id:1, name:'Tayab' }
Column	Ek field	id, name, email
Primary Key	Unique identifier	id SERIAL PRIMARY KEY
Foreign Key	Doosri table ka reference	userId INT REFERENCES User(id)
Index	Fast search ke liye	CREATE INDEX ON User(email)

■ Useful psql Commands

PSQL TERMINAL

```
psql -U postgres # login karo
\l # saare databases list karo
\c taskmanager # database select karo
\dt # saari tables dekho
\d Task # ek table ka structure dekho
\q # quit karo

-- SQL directly bhi likh sakte ho
SELECT * FROM "Task";
SELECT * FROM "User" WHERE id = 1;
```

NOTE Production mein direct psql se data mat churao — Prisma Studio ya proper admin tool use karo.

2. Prisma ORM — Kya hai, kyun use karte hain

ORM (Object Relational Mapper) ka matlab hai — tum JavaScript objects se database se baat karte ho, SQL likhne ki zaroorat nahi. Prisma specifically type-safe hai — galat query likhoge toh error pehle hi aa jaega.

Bina Prisma (Raw SQL)	Prisma ke saath
SELECT * FROM "Task" WHERE id = 1	prisma.task.findUnique({ where: { id: 1 } })
INSERT INTO "Task" (title) VALUES ('test')	prisma.task.create({ data: { title: 'test' } })
UPDATE "Task" SET completed=true WHERE id=1	prisma.task.update({ where:{id:1}, data:{completed:true} })
DELETE FROM "Task" WHERE id = 1	prisma.task.delete({ where: { id: 1 } })

■ Prisma ke 3 Main Parts

PRISMA ARCHITECTURE

1. Prisma Schema → prisma/schema.prisma → models define karo
2. Prisma Migrate → npx prisma migrate → DB mein tables banao
3. Prisma Client → @prisma/client → queries chalao

3. Project Setup — Zero se Shuru

TERMINAL — FRESH SETUP

```
# Folder banao
mkdir my-project && cd my-project
npm init -y

# Packages install karo
npm install express @prisma/client dotenv
npm install prisma --save-dev

# Prisma initialize karo
npx prisma init

# .env mein DATABASE_URL set karo
DATABASE_URL="postgresql://postgres:PASSWORD@localhost:5432/mydb?schema=public"
```

File	Kaam
prisma/schema.prisma	Models aur relations define karo yahan
.env	Database URL aur secrets — kabhi GitHub pe mat daalna
.env.example	Template file — values ke baghair — commit karo
node_modules/	Kabhi commit mat karo — .gitignore mein daalo

4. Schema aur Models — Blueprint Banana

PRISMA SCHEMA

```
// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url = env("DATABASE_URL")
}

model User {
  id Int @id @default(autoincrement())
  email String @unique
  name String?
  password String
  tasks Task[]
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  deletedAt DateTime? // soft delete pattern
}

model Task {
  id Int @id @default(autoincrement())
  title String
  description String?
  completed Boolean @default(false)
  userId Int?
  user User? @relation(fields: [userId], references: [id])
  createdAt DateTime @default(now())
}
```

```
}
```

Attribute	Matlab
@id	Primary key hai ye field
@default(autoincrement())	Auto 1, 2, 3... assign hota hai
@unique	Ye value repeat nahi ho sakti (jaise email)
String?	? = optional field, null ho sakta hai
@default(now())	Current timestamp automatically save
@updatedAt	Har update pe automatically time update
@relation(...)	Foreign key relation define karta hai
@@index([field])	Search fast karne ke liye index lagao

5. Migrations — DB Changes Safely Apply Karna

Migration ek version history hai tumhare database ka. Har schema change ke saath ek migration file banti hai jo exact SQL store karti hai.

MIGRATION COMMANDS

```
# LOCAL DEVELOPMENT — schema change ke baad
npx prisma migrate dev --name add_user_table
npx prisma migrate dev --name add_phone_to_user

# PRODUCTION SERVER — sirf pending migrations apply karo
npx prisma migrate deploy

# Migration status check karo
npx prisma migrate status

# Local DB reset karo (DANGER — sab data jaega)
npx prisma migrate reset

# Prisma client regenerate karo
npx prisma generate
```

DIKKAT 'migrate dev' KABHI production pe mat chalana — ye dev-only command hai. Production pe sirf 'migrate deploy' chalao.

TIP Migration files (prisma/migrations/) ko ZAROOR git commit karo — team ka DB tabhi sync rahega.

6. Prisma CRUD Queries — Complete Reference

■ CREATE

CREATE

```
// Single record banao
const user = await prisma.user.create({
  data: { name: 'Tayab', email: 'tayab@test.com', password: 'hashed' }
})

// Multiple records ek saath banao
await prisma.task.createMany({
  data: [
    { title: 'Task 1', userId: 1 },
    { title: 'Task 2', userId: 1 },
  ]
})
```

■ READ

READ

```
// Saare records
await prisma.user.findMany()

// Ek record by ID
await prisma.user.findUnique({ where: { id: 1 } })

// Sirf kuch fields lo (password hide karo)
await prisma.user.findMany({
  select: { id: true, name: true, email: true }
})

// Related data saath fetch karo (JOIN)
await prisma.user.findUnique({
  where: { id: 1 },
  include: { tasks: true }
})

// Filter karo
await prisma.task.findMany({
  where: { completed: false, userId: 1 }
})
```

■ UPDATE & DELETE

UPDATE & DELETE

```
// Update karo
await prisma.task.update({
  where: { id: 1 },
  data: { completed: true }
})

// Delete karo
await prisma.task.delete({ where: { id: 1 } })

// Upsert – exist kare toh update, warna create
await prisma.user.upsert({
  where: { email: 'tayab@test.com' },
  update: { name: 'Updated' },
  create: { name: 'Tayab', email: 'tayab@test.com', password: 'x' }
})
```

7. Express + Prisma — REST API Banana

■ PrismaClient Singleton — src/prisma.js

```
src/prisma.js
// src/prisma.js
const { PrismaClient } = require('@prisma/client')
// Singleton pattern – ek hi instance banao
// Warna dev mode mein har restart pe naya connection banega
const globalForPrisma = global
const prisma = globalForPrisma.prisma ?? new PrismaClient()
if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma
module.exports = prisma
```

■ REST Endpoints Pattern

Method	Endpoint	Kaam	Prisma Query
GET	/tasks	Saare tasks	findMany()
GET	/tasks/:id	Ek task	findUnique()
POST	/tasks	Naya task banao	create()
PATCH	/tasks/:id	Task update karo	update()
DELETE	/tasks/:id	Task delete karo	delete()

8. MVC Architecture — Production Folder Structure

```
FOLDER STRUCTURE
src/
  ■■■ index.js # Server setup, middleware, error handler
  ■■■ prisma.js # PrismaClient singleton
  ■■■ routes/
    ■ ■■■ tasks.js # Endpoints define karo (router.get, router.post)
    ■ ■■■ users.js
  ■■■ controllers/
    ■ ■■■ tasks.js # Business logic yahan (req/res handle)
    ■ ■■■ users.js
  ■■■ middlewares/
    ■ ■■■ auth.js # JWT verify middleware
    ■ ■■■ validate.js # Input validation middleware
  ■■■ services/ # Complex logic alag karo (optional)
    ■ ■■■ emailService.js
  ■■■ utils/
  ■■■ helpers.js # Reusable functions
```

File	Responsibility	Example
routes/tasks.js	Sirf endpoints define karo	router.get('/', getAllTasks)
controllers/tasks.js	Logic likho, req/res handle karo	const tasks = await prisma.task.findMany()
middlewares/auth.js	Request interceptor	verify JWT token
services/	Business logic jo controller se bari ho	Email bhejna, payment process

9. Error Handling — Production Grade

ERROR HANDLING PATTERN

```
// src/index.js – Global Error Handler (sabse neeche hona chahiye)
app.use((err, req, res, next) => {
  console.error(err) // server logs mein print karo

  // Prisma specific errors
  if (err.code === 'P2025') {
    return res.status(404).json({ message: 'Record nahi mila' })
  }
  if (err.code === 'P2002') {
    return res.status(400).json({ message: 'Ye value already exist karti hai' })
  }
  if (err.code === 'P2003') {
    return res.status(400).json({ message: 'Related record nahi mila' })
  }
  res.status(500).json({ message: 'Server error' })
})

// Controller mein try/catch + next(err)
const getTask = async (req, res, next) => {
  try {
    const task = await prisma.task.findUnique({ where: { id: Number(req.params.id) } })
    if (!task) return res.status(404).json({ message: 'Task nahi mila' })
    res.json(task)
  } catch (err) {
    next(err) // global handler ko bhejo
  }
}
```

Prisma Error Code	Matlab
P2002	Unique constraint fail – email already exists
P2025	Record not found — delete/update pe id nahi mila
P2003	Foreign key fail – related record nahi hai
P2000	Value too long — column size exceed
P2011	Null constraint – required field missing

10. Input Validation — Security Layer

Validation zaroor karo — user ka data kabhi trust mat karo. Bina validation ke koi empty string ya null Prisma mein jaegi aur DB corrupt ho sakta hai.

INSTALL

```
# Zod install karo – best validation library
npm install zod
```

ZOD VALIDATION

```
// src/middlewares/validate.js
const { z } = require('zod')

const taskSchema = z.object({
  title: z.string().min(1, 'Title required hai').max(100),
```

```

description: z.string().optional(),
})

const validateTask = (req, res, next) => {
const result = taskSchema.safeParse(req.body)
if (!result.success) {
return res.status(400).json({ errors: result.error.errors })
}
req.body = result.data // sanitized data
next()
}

module.exports = { validateTask }

// Route mein use karo:
router.post('/', validateTask, createTask)

```

11. Relations — One-to-Many, Many-to-Many

■ One-to-Many — Ek User ke bahut saare Tasks

```

ONE-TO-MANY
model User {
id Int @id @default(autoincrement())
tasks Task[] // ek user ke multiple tasks
}

model Task {
id Int @id @default(autoincrement())
userId Int
user User @relation(fields: [userId], references: [id])
}

// Query – user ke saath tasks bhi lao
await prisma.user.findUnique({
where: { id: 1 },
include: { tasks: true }
})

```

■ Many-to-Many — Posts aur Tags

```

MANY-TO-MANY
model Post {
id Int @id @default(autoincrement())
tags Tag[]
}

model Tag {
id Int @id @default(autoincrement())
name String @unique
posts Post[]
}

// Prisma automatically junction table banata hai

```

12. Advanced Prisma — Filtering, Sorting, Pagination

ADVANCED QUERIES

// FILTERING

```
await prisma.task.findMany({
  where: {
    completed: false,
    title: { contains: 'urgent', mode: 'insensitive' },
    createdAt: { gte: new Date('2026-01-01') }
  }
})
```

// SORTING

```
await prisma.task.findMany({
  orderBy: { createdAt: 'desc' }
})
```

// PAGINATION

```
await prisma.task.findMany({
  skip: (page - 1) * 10, // page 1 = skip 0, page 2 = skip 10
  take: 10, // 10 records per page
  orderBy: { id: 'desc' }
})
```

// COUNT

```
const total = await prisma.task.count({ where: { completed: false } })
```

// TRANSACTION — dono operations ya toh hongii ya koi nahi

```
await prisma.$transaction([
  prisma.task.create({ data: { title: 'Task' } }),
  prisma.user.update({ where: { id: 1 }, data: { name: 'Updated' } })
])
```

13. Authentication — JWT + Prisma

INSTALL

```
npm install bcryptjs jsonwebtoken
```

JWT AUTH PATTERN

// controllers/auth.js

```
const bcrypt = require('bcryptjs')
const jwt = require('jsonwebtoken')
const prisma = require('../prisma')

// REGISTER
const register = async (req, res, next) => {
  try {
    const { name, email, password } = req.body
    const hashed = await bcrypt.hash(password, 10)
    const user = await prisma.user.create({
      data: { name, email, password: hashed }
    })
    res.json({ message: 'User ban gaya' })
  } catch (err) { next(err) }
}

// LOGIN
const login = async (req, res, next) => {
```

```

try {
  const { email, password } = req.body
  const user = await prisma.user.findUnique({ where: { email } })
  if (!user) return res.status(401).json({ message: 'Invalid credentials' })
  const valid = await bcrypt.compare(password, user.password)
  if (!valid) return res.status(401).json({ message: 'Invalid credentials' })
  const token = jwt.sign({ userId: user.id }, process.env.JWT_SECRET, { expiresIn: '7d' })
  res.json({ token })
} catch (err) { next(err) }
}

// AUTH MIDDLEWARE
const authMiddleware = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1]
  if (!token) return res.status(401).json({ message: 'Token nahi hai' })
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET)
    req.userId = decoded.userId
    next()
  } catch { res.status(401).json({ message: 'Invalid token' }) }
}

```

14. Environment Variables — Best Practices

.env FILES

```

# .env — actual values (KABHI commit mat karo)
DATABASE_URL="postgres://postgres:mypassword@localhost:5432/mydb?schema=public"
JWT_SECRET="super-secret-key-yahan-likho"
PORT=3000
NODE_ENV=development

# .env.example — template (commit karo — values empty)
DATABASE_URL=
JWT_SECRET=
PORT=
NODE_ENV=

```

USAGE

```

// Code mein use karo
const PORT = process.env.PORT || 3000
const JWT_SECRET = process.env.JWT_SECRET

// .gitignore mein ZARoor daalo
.env
node_modules/
generated/

```

15. Interview Preparation — PostgreSQL, Prisma & Backend Q&A;

PostgreSQL Questions

Q: PostgreSQL aur MySQL mein kya farq hai?

A: PostgreSQL zyada advanced hai — JSON support, full-text search, complex queries better handle karta hai. MySQL simpler hai aur thoda faster simple read-heavy apps mein. Production grade apps mein PostgreSQL prefer hota hai.

Q: Primary Key aur Foreign Key mein kya farq hai?

A: Primary Key ek table mein unique identifier hai (jaise user ka id). Foreign Key doosri table ka reference hai — jaise Task table mein userId, User table ke id ko point karta hai. Ye tables ke beech relation banata hai.

Q: Index kya hota hai aur kab use karte hain?

A: Index ek shortcut hai — jaise book ka index. Bina index ke PostgreSQL poori table scan karta hai. Jis column pe frequently WHERE ya JOIN lagao, uspe index lagao. Prisma mein `@@index([email])` se lagta hai.

Q: Transaction kya hota hai?

A: Transaction mein multiple operations ek saath hoti hain — ya toh saari hogi ya koi nahi. Jaise bank transfer — paise ek account se kate aur doosre mein aaye, dono hone chahiye warna rollback.

Prisma Questions

Q: Prisma ORM kya hai aur kyun use karte hain?

A: Prisma ek type-safe ORM hai jo SQL likhne ki zaroorat khatam karta hai. Schema define karo, migrate karo, aur JavaScript se direct database queries chalao. Type safety ki wajah se runtime errors kam hote hain.

Q: migrate dev aur migrate deploy mein kya farq hai?

A: migrate dev local development ke liye hai — naya migration file banata hai aur apply karta hai. migrate deploy production ke liye hai — sirf pending migrations apply karta hai, koi naya file nahi banata. Production pe sirf migrate deploy chalao.

Q: N+1 problem kya hota hai Prisma mein?

A: Jab ek query ke baad loop mein aur queries chalao. Jaise 10 users fetch karo, phir har user ke liye alag query chalao tasks ke liye = 11 queries. Solution: include use karo — ek hi query mein sab aa jaega.

Q: Prisma mein select aur include mein kya farq hai?

A: select: sirf specific fields lo (jaise password chupaana). include: related models bhi saath lao (jaise user ke tasks). Dono ek saath use ho sakti hain.

Q: Soft delete kya hota hai?

A: Record actually delete nahi karte, balki deletedAt field mein timestamp store karte hain. Fir queries mein where: { deletedAt: null } filter lagate hain. Benefit: data recover ho sakta hai baad mein.

Express & Node.js Questions

Q: Middleware kya hota hai Express mein?

A: Middleware ek function hai jo request aur response ke beech mein chalta hai. Jaise auth middleware — pehle token verify karo, phir controller ko bhejo. `app.use()` se globally lagta hai, ya specific route pe.

Q: MVC pattern kya hai?

A: Model-View-Controller. Routes request receive karti hain, Controllers logic handle karte hain, Models data se deal karte hain. Isse code organize rehta hai, team mein alag log alag layers pe kaam kar sakte hain.

Q: async/await aur Promises mein kya farq hai?

A: Promises `.then().catch()` se handle hoti hain — chain ban jaati hai jo padhi nahi jaati. `async/await` usi kaam ko synchronous jaisa likhne deta hai — code clean aur readable hota hai. Internally dono same kaam karte hain.

Q: REST API mein HTTP status codes kaunse important hain?

A: 200 OK, 201 Created (POST ke baad), 400 Bad Request (validation fail), 401 Unauthorized (token nahi), 403 Forbidden (permission nahi), 404 Not Found, 500 Internal Server Error. Sahi status code dena API ka contract hota hai.

Q: JWT (JSON Web Token) kaise kaam karta hai?

A: Login pe server ek token generate karta hai jisme user ID hoti hai — sign karke client ko deta hai. Agle requests mein client ye token Authorization header mein bhejta hai. Server verify karta hai token real hai ya nahi — database query ki zaroorat nahi.

Interview tip: Sirf theory mat batao — apna project example do. 'Maine Task Manager mein Prisma ka N+1 problem fix kiya include() se' — ye answer interviewer ko impress karta hai.